ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

ΣΧΟΛΗ ΕΠΙΣΤΗΜΩΝ & ΤΕΧΝΟΛΟΓΙΑΣ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ
SCHOOL OF INFORMATION SCIENCES & TECHNOLOGY
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
DEPARTMENT OF INFORMATICS

Natural Language Processing

Text Classification &
POS Tagging with RNNs

Dec 2025

Maria Schoinaki

Electra Papadopoulou

Leonidas Kontogiannis

You can view the full code implementation for the **text classification** in the following link:

You can view the full code implementation for the **POS tagger** in the following link:

All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment.

Tweet Sentiment Classification with RNNs

For this assignment we used the Twitter Sentiment Analysis dataset from Kaggle. This dataset consists of ~100.000 tweets (sentiment texts), the id of each tweet, and the sentiment (0 or 1), and it was used as the basis for training (70%), development (15%), and testing (15%) of the sentiment classifiers. The same dataset had been used for previous assignments**,** thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report here.

Data Preprocessing/ Cleaning

As shown previously, the dataset is roughly balanced (≈43.5% negative, ≈56.5% positive), and the tweets are short (mean ≈ 13 words), which is typical for Twitter.
In order to preprocess the dataset, we implement **spaCy** and **regular expressions.**

- **Text Cleaning:**
  For a dataset full of tweets, it is important to scrap the information that is not useful for sentiment classification, such as **symbol-prefixed words**; we eliminate mentions, tagging, quoting, HTML

entities and URLs.. Non-word characters and extra spaces are also removed and lowercasing is applied to ensure there's case consistency across tokens.

- **Linguistic Normalization:**

  To preprocess each tweet, we will utilize **Part-Of-Speech (POS)** tagging, specifically spaCy's **English language model** (*en_core_web_sm*). Only **tokenization** and **lemmatization** were required.

The result of the process is a list of tweets. Here are some examples of the results:

Original tweet text:
@akiraLOVE haha, yeah hey, he replied to you too right??

Filtered tweet text:
haha yeah hey he reply to you too right

Original tweet text:
@ashers11 Whyy?

Filtered tweet text:
whyy

Embedding Construction For the RNN

In contrast to the earlier MLP baseline, where documents were represented using TF-IDF vectors reduced via SVD, here we adopt learned **word embeddings** for the **Bi-LSTM** model. Word embeddings allow us to exploit syntactic and semantic structure and provide a more suitable input representation for recurrent neural networks.

After applying the text-cleaning pipeline (*see preprocessing section*), we constructed a **vocabulary of 34,197 unique tokens**, augmented with two special tokens, <pad> and <unk>.

A vocabulary of this size empirically yielded stronger performance than pruning rare words. During model training, we allow the embedding layer to be fine-tuned, enabling the network to adapt the learned representations to the classification objective.

Although the majority of tweets are valid, a small number become empty after cleaning. These empty entries were removed prior to training Word2Vec to prevent degenerate sequences. The final Word2Vec model trained successfully and produced reliable word vectors for all **34,197 unique tokens**.

We trained **skip-gram Word2Vec embeddings** directly on the cleaned training corpus using the following configuration: **embedding size 100**, **window size 5**, **min_count=1**, and **10 epochs**. This yielded dense distributional representations for every token in the vocabulary. To integrate these embeddings into the neural model, we constructed an **embedding_matrix** in which all in-vocabulary tokens received their learned Word2Vec vector, while the two special tokens were initialized with small random values. Then we create a PyTorch **nn.Embedding** layer with **requires_grad = True** in order to fine-tune them in the model training.

Tweets are then converted into sequences of vocabulary indices and packed via PyTorch DataLoaders using a custom **collate_fn** that pads variable-length sequences. This creates efficient batches suitable for training the **Bi-LSTM** architecture while preserving the sequential nature of the data. The overall pipeline: cleaning, vocabulary construction, Word2Vec training, embedding initialization, and padded batching, ensures that each model receives consistent, information-rich sequence representations.

Experimental Setup

For Exercise 1 we reuse exactly the same dataset, splits, and preprocessing pipeline as in Exercise 11 (Part 2) and Exercise 4 (Part 3):

- **Corpus:** twitter-sentiment-analysis2 (Kaggle), with ~100K tweets labeled with binary sentiment (0 = Negative, 1 = Positive).
- **Splits:** 70% train, 15% development, 15% test, stratified by label, using random_state=2025.
- **Preprocessing:** Text Cleaning and Linguistic Normalization, as described above.

Baseline Classifiers

We consider two baselines, as requested:

TF-IDF + Logistic Regression (best model from Exercise 11 part 2):

- Feature selection: SelectKBest(mutual_info_classif, k=30,000)
- Classifier: LogisticRegression with C=1 (best value found in Exercise 11)

We reuse this tuned **TF-IDF + LR** as our strong linear baseline and retrain it on the same train/dev/test split used for the MLP.

MLP with Truncated TF-IDF SVD vectors (From Exercise 4 in part 3):

- Input layer: 500-dimensional SVD features.
- Hidden layers: a single fully connected layer with:
    - hidden_sizes = [512]
    - Activation: ReLU
    - Dropout: 0.7
- Output layer: linear layer with 2 units (Negative, Positive), trained with cross-entropy loss.

Additionally, we have included a **majority baseline** classifier in our comparisons that serves as a lower bound for performance. The implementation was via scikit-learn's Dummy Classifier and it was trained on labels only.

Model Architecture

Network architecture

The final best configuration (after tuning 8 random configurations) is:

- **Input layer:** token indices mapped to pretrained embeddings (fine-tuned), embedding dropout 0.2.
- **Sequence encoder:** 2-layer bidirectional LSTM with 128 hidden units per direction, inter-layer dropout 0.2.
- **Attention layer:** single dense self-attention over BiLSTM hidden states to produce a context vector.
- **Classifier head:** linear layer reducing hidden dimension by half → ReLU → dropout 0.2 → final linear layer with 2 outputs (Negative, Positive), trained with cross-entropy loss.

Overall, the model captures sequential dependencies in the tweet text, while attention focuses on the most informative tokens for classification. The BiLSTM provides contextualized representations, and the attention mechanism aggregates them for robust prediction.

Data loaders

Tweets are tokenized and converted to index sequences with **<pad>** and **<unk>** tokens. Variable-length sequences are padded to the batch maximum.

- Batch size: 256
- Training loader shuffled, dev/test loaders not shuffled

This allows efficient mini-batch training using packed sequences, maintaining proper handling of variable-length inputs.

Training

The model is trained using mini-batches of tokenized sentences. Each epoch performs a full pass over the training set, computing the forward pass, loss, and gradients, followed by an optimizer update. We use Adam with optional weight decay, cross-entropy loss, and gradient clipping to stabilize LSTM training. After every epoch, the model is evaluated on both the training and development sets, computing accuracy, loss, and Macro-F1. Early stopping is applied based on development Macro-F1: if the score does not improve for several epochs (**patience = 5**), training stops automatically. The best model parameters are tracked and restored at the end, ensuring that evaluation always uses the best-performing checkpoint rather than the final epoch.

Hyperparameter Tuning (Random Search)

We perform **random search** over a structured hyperparameter space. Random search is chosen over exhaustive grid search because grid search becomes inefficient when multiple hyperparameters interact non-linearly. Random sampling explores a larger variety of configurations at a fraction of the computational cost.

Training Procedure

For each sampled configuration:

1. A new BiLSTM-SelfAttention model is created using a random sample of hyperparameters.
2. The model is trained for up to **10 epochs** with **early stopping** (patience=5).
3. After training, the **Macro F1-score on the development set** is used as the selection metric.
4. The **best-performing model**, its hyperparameters, and its full training history are stored.

Best Model and Hyperparameters

After evaluating **8** random configurations, the best model was obtained with the following hyperparameters:

- hidden_size = 128
- num_layers = 2
- dropout = 0.2
- attn_hidden_dim = single dense attention
- learning rate = 0.01
- fine_tune_embeddings = True

The **best model's macro F1 score** on the development set was ~**0.76**.

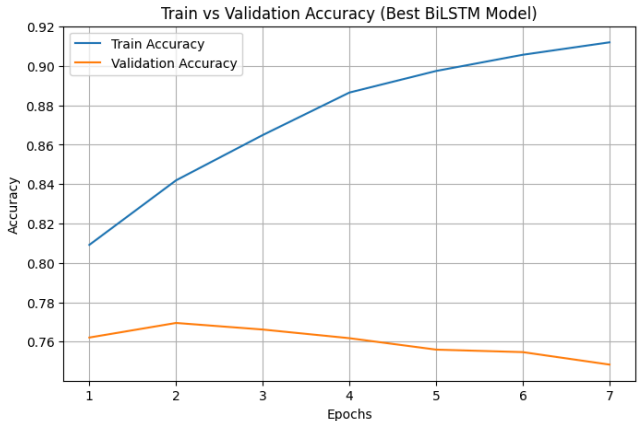
Notes on Implementation

- We ensured **no duplicate configurations** were sampled during the random search.
- The training loop returned a full **history** containing: train_loss, dev_loss, train_accuracy, val_accuracy, train_f1, and val_f1 for each epoch, enabling detailed learning curve analysis.

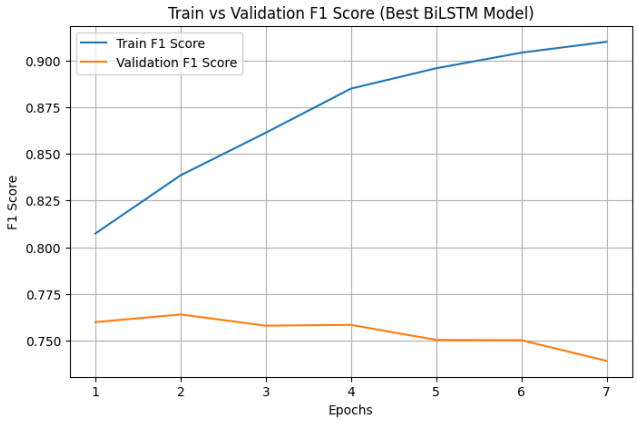
Loss Curves

After training, we analyze the behavior of the best model across epochs by visualizing the training history:

- **Train vs. Dev Loss:** The curves help determine whether the model continues learning **(training loss decreasing)**, whether it generalizes well **(dev loss follows train loss without diverging)** and whether overfitting occurs **(dev loss increases while train loss continues to drop).**



- **Train vs. Validation Accuracy:** We want to check if the model is consistently improving its predictions.



- **Train vs. Validation F1 Score:** It monitors whether the model improves in its ability to handle all classes consistently, and whether precision–recall tradeoffs change over training, though it is more informative when classes have uneven representation.

Additionally, all of these curves can show when the model starts overfitting, which is happening in this case, even though we have tuned our model to the best epoch. The dev loss does not follow the train loss, and thus the validation accuracy and F1-score do not improve that greatly over time.

Evaluation

We report per-class **Precision (P)**, **Recall (R)**, **F1**, and **PR-AUC** and we also include **Macro-Averaged P/R/F1/PR-AUC**. Models are **trained** on ***train*** dataset, **tuned** in ***dev*** dataset and **evaluated** in ***test*** dataset.

Systems Compared

- **Majority baseline** (*dummy classifier-most_frequent*): Always predicts the training set's majority class.
- **LR** (*TF-IDF*) **tuned**: Logistic Regression with TF-IDF word n-grams (*uni, bi, tri*).
- **MLP** (*TF-IDF*) **tuned**: Multi-Layered Perceptron with SVD features.
- **BiLSTM** (word embedding) **tuned**: Bidirectional LSTM using pretrained word embeddings.

Results

We provide three tables (*train, dev, test*). Each table has one row per model and four columns per class (*P, R, F1, PR-AUC*), followed by *Macro-P, Macro-R, Macro-F1, Macro-PR-AUC.*

Training Set

=== TRAIN RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435364	0.564636	1.000000	0.721747	0.564636	0.282318	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.793206	0.827645	0.810060	0.878431	0.862502	0.833629	0.847820	0.930190	0.827854	0.830637	0.828940	0.904310
2	MLP	0.830395	0.701661	0.760619	0.867907	0.794526	0.889499	0.839334	0.910626	0.812460	0.795580	0.799977	0.889266
3	BiLSTM	0.830777	0.799881	0.815036	0.890813	0.849938	0.874320	0.861957	0.937867	0.840357	0.837101	0.838496	0.914340

- The **majority baseline** reaches Macro-F1 ≈ 0.36 and Macro PR-AUC ≈ 0.50 (*constant prediction yields poor F1 but PR-AUC near the class prior*).
- The **BiLSTM** model achieves a high training score that surpasses our MLP and LR Models, which it is also expected, as our model is more **word-order aware.**

Development Set

=== DEV RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435391	0.564609	1.000000	0.721725	0.564609	0.282304	0.500000	0.360863	0.500000
1	TF-IDF+LR	0.722174	0.757121	0.739234	0.805588	0.805447	0.775390	0.790132	0.876435	0.763810	0.766255	0.764683	0.841011
2	MLP	0.709568	0.608729	0.655292	0.746550	0.728076	0.807865	0.765898	0.826373	0.718822	0.708297	0.710595	0.786461
3	BiLSTM	0.748343	0.712308	0.729881	0.810292	0.785796	0.815017	0.800140	0.881620	0.767070	0.763662	0.765010	0.845956

- **Baseline:** Macro F1≈0.3609, Macro PR-AUC=0.5000
- **LR (*TF-IDF*):** P=0.7638, R=0.7666, F1=0.7646, PR-AUC=0.8410
- **MLP (*TF-IDF*)**: Macro F1=0.7134, Macro PR-AUC=0.7865
- **BiLSTM(world embeddings):** Macro F1=0.765, Macro PR-AUC=0.8459

Observation: BiLSTM clearly dominates the MLP and is close to TF-IDF LR on Dev (≈+0.1 Macro-F1, +0.04 PR-AUC). The baseline is far behind, as expected.

Test Set

=== TEST RESULTS ===

	Model	P(0)	R(0)	F1(0)	PR-AUC(0)	P(1)	R(1)	F1(1)	PR-AUC(1)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.000000	0.000000	0.000000	0.435362	0.564638	1.000000	0.721749	0.564638	0.282319	0.500000	0.360874	0.500000
1	TF-IDF+LR	0.726814	0.750077	0.738262	0.810192	0.802421	0.782619	0.792396	0.875979	0.764618	0.766348	0.765329	0.843085
2	MLP	0.713409	0.605360	0.654958	0.753251	0.727532	0.812493	0.767669	0.825070	0.720471	0.708926	0.711314	0.789160
3	BiLSTM	0.750205	0.704913	0.726854	0.813666	0.782766	0.819174	0.800557	0.879238	0.766486	0.762044	0.763705	0.846452

- **Baseline:** Very poor for class 0 (F1=0), okay for class 1; overall macro F1 is low (0.36).
- **TF-IDF + LR:** Solid balanced performance, macro F1 ~0.765, strong precision and recall for both classes and PR-AUC ~ 0.843
- **MLP:** Worse than TF-IDF+LR and **BiLSTM**, macro F1 ~0.71.
- **BiLSTM:** Close to the **TF-IDF + LR**, balanced precision and recall across both classes; macro F1 ~0.763, slightly worse than TF-IDF+LR, and PR-AUC ~0.846 slightly better.

Takeaway: The BiLSTM model with self-attention achieves close performance with the **TF-IDF + LR.**

POS Tagging

For this assignment we used the **UD Ancient Greek PROIEL** treebank from Universal Dependencies. The corpus contains morphologically and syntactically annotated **Ancient Greek** texts. After loading the dataset, we obtained **15,016** sentences for training, **1,019** sentences for development, and **1,047** sentences for testing. Each sentence consists of a sequence of tokens paired with their corresponding Universal POS tags (UPOS). The same dataset had been used for previous assignments**,** thus we will be brief in summarizing the preprocess and stage. For more details, you can read the report here.

Data Preprocessing

For the MLP model, we convert each sentence into overlapping windows of 5 tokens: the target word together with its two preceding and two following words. At sentence boundaries, missing context is filled with the special <PAD> token.

The following table summarizes **key statistics** for the training, development, and test subsets of the **Ancient Greek PROIEL** treebank:

```
TRAIN stats:
  sentences:        15016
  tokens:           187039
  avg sent length:  12.46
  median sent len:  10.00
  vocab size:       30349
  number of tags:   13
  most common tags:
    VERB: 34517
    NOUN: 31424
    DET: 28070
    ADV: 20288
    PRON: 19756
    ADP: 13812
    CCONJ: 12117
    ADJ: 10361
    PROPN: 7745
    SCONJ: 3610
```

```
DEV stats:
  sentences:        1019
  tokens:           13652
  avg sent length:  13.40
  median sent len:  11.00
  vocab size:       4541
  number of tags:   13
  most common tags:
    VERB: 2671
    NOUN: 2185
    DET: 2082
    PRON: 1446
    ADV: 1378
    CCONJ: 978
    ADP: 922
    ADJ: 729
    PROPN: 618
    SCONJ: 257
```

```
TEST stats:
  sentences:        1047
  tokens:           13314
  avg sent length:  12.72
  median sent len:  11.00
  vocab size:       4495
  number of tags:   13
  most common tags:
    VERB: 2494
    NOUN: 2209
    DET: 2006
    PRON: 1427
    ADV: 1416
    CCONJ: 1011
    ADP: 885
    ADJ: 752
    PROPN: 494
    AUX: 248
```

Word Embeddings

We trained **Word2Vec** embeddings directly on the **training corpus**. Only the word sequences were used as input for Word2Vec, and we employed a **skip-gram architecture** with a vector dimension of **100**, a window size of 5, and a minimum frequency threshold of one. The Word2Vec model was trained for ten epochs. This produced distributional word vectors for all words seen in the training data.

Our Model

The POS tagger is a **sequence-based bidirectional LSTM (BiLSTM)** model that operates over entire sentences rather than fixed windows. Each word index is mapped to a continuous vector through an embedding layer, which is initialized with pretrained Word2Vec embeddings and fine-tuned during training. The embedded sentence is then processed by a multi-layer BiLSTM, allowing the model to capture both past and future context for every position in the sequence. The concatenated forward and backward hidden states are passed through a dropout layer and finally through a linear classifier that predicts a UPOS tag for each word in the sentence. This architecture enables the model to incorporate long-range contextual dependencies alongside pretrained lexical information.

Training used mini-batches of padded sentences, optimized with Adam and cross-entropy loss (ignoring padded positions). After each epoch, model performance was evaluated on the development set using loss and macro-F1. Early stopping was employed based on dev-set loss, and the parameters corresponding to the best macro-F1 score were restored at the end of training, ensuring good generalization and preventing overfitting.

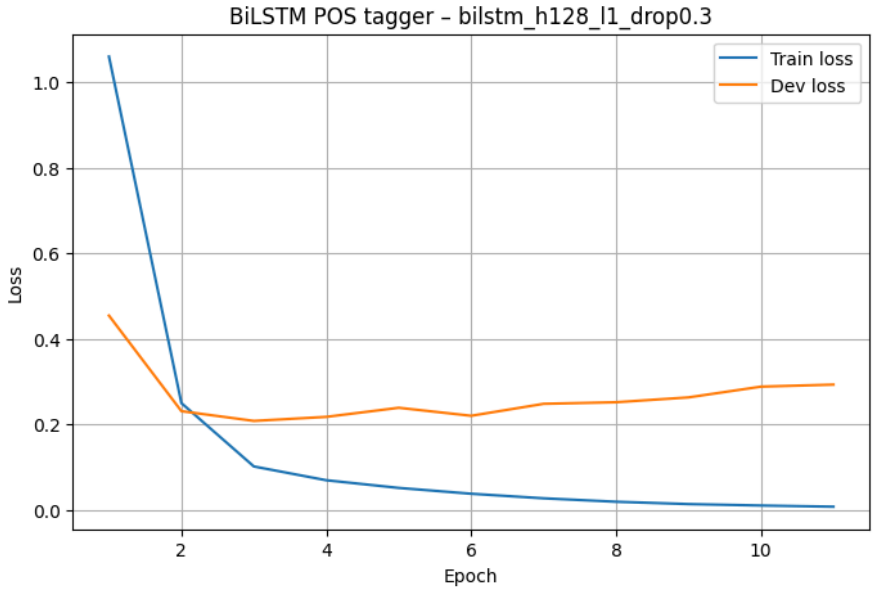
Fine Tuning and Training

To identify the best sequence model architecture, we conducted a hyperparameter search over several BiLSTM configurations, varying the number of hidden layers, their sizes, dropout rate and learning rate. Each configuration was trained on the training set and evaluated on the development set using macro-F1 as the early-stopping and model-selection criterion. For every run, we recorded the training and development loss curves to monitor optimization behavior and generalization. After all models were trained independently, the configuration achieving the highest macro-F1 on the development set was selected as the final model. Our best-performing model is a single-layer BiLSTM with 128 hidden units and a dropout rate of 0.3.

We train the **final BiLSTM** with:

- Hidden sizes: [128]
- Dropout: best value from dropout tuning (0.3)
- Epochs: best value from epoch tuning (20 in our run)

- Optimizer: Adam(lr=1e-3)



The training loss decreases monotonically, while the development loss initially decreases and then rises again, indicating overfitting if we were to train for too many epochs. However, by selecting the epoch with the best development macro-F1, the final model corresponds to an early epoch before severe overfitting sets in.

Character-Aware BiLSTM Model

In addition to the word-level BiLSTM, we implemented a **character-aware BiLSTM tagger** that adds subword information to each word representation. Similarly to our BiLSTM above, it uses pretrained Word2Vec embeddings as input, but it additionally learns trainable character embeddings and a character-level BiLSTM that processes each word as a sequence of characters. The final character representation is combined with the word embedding and fed into the word-level BiLSTM. This allows the model to encode morphological features, handle rare and misspelled words, and generalize to unseen forms more effectively. Training, evaluation and hyperparameter search followed the same procedure as for the original BiLSTM, though we also added whether to include the character-level encoder as a parameter.

Our reasoning for adding this extended architecture is that by introducing subword modeling, the new character-aware BiLSTM does not only rely on learned word embeddings, but decomposes each word into its characters and can thus handle more morphological regularities, as well as OOV words.

BiLSTM with Temporal Averaging

To improve model stability and reduce variance across training epochs, we also implemented **temporal weight averaging**. For the best configuration, the model was trained as before, but this time simultaneously maintaining a running average of all parameter values across epochs. After training, we evaluate both the best single checkpoint and the temporally averaged model on the dev set. We test whether smoothing the parameter trajectory leads to better generalization. Similarly to our baseline approach, we keep track of loss curves for every configuration and compare macro-F1 scores.

Evaluation

Once again, we create three tables that summarize the training, test and dev set performance of all our POS-tagging models across all UPOS categories. The results that interest us are as follows:

- On the **train set, MLP (best config) > BiLSTM-TA > BiLSTM > BiLSTM-char > Baseline**.
- On the **dev set, BiLSTM-char > BiLSTM-TA ≈ BiLSTM > MLP > Baseline** in terms of Macro-F1 and Macro-PR-AUC. The per-word baseline still performs reasonably on frequent tags but struggles with rarer ones.
- The char-aware BiLSTM leverages subword information to slightly boost performance. Standard BiLSTM and temporal-averaged BiLSTM (BiLSTM-TA) perform very close to each other, with the MLP lagging slightly behind, suggesting that sequence modeling helps generalization on the dev set.
- On the **test set,** the trend continues: **BiLSTM-char slightly outperforms others**, followed closely by BiLSTM-TA and BiLSTM. The MLP remains competitive but slightly lower in Macro-F1. The baseline consistently underperforms for less frequent tags.

Overall, sequence models with BiLSTMs, especially those incorporating characters or temporal averaging, achieve the best generalization across all UPOS tags.

=== TRAIN RESULTS ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9845	0.9908	0.9876	0.9759	0.9765	0.9940	0.9852	0.9711	0.9726	...	0.8797	0.7775	0.9996	0.9992	0.9994	0.9989	0.9671	0.9665	0.9664	0.9384
1	MLP (best config)	0.9977	0.9965	0.9971	0.9999	0.9979	0.9997	0.9988	1.0000	0.9980	...	0.9942	0.9998	0.9998	1.0000	0.9999	1.0000	0.9974	0.9972	0.9973	0.9999
2	BiLSTM	0.9928	0.9943	0.9935	0.9998	0.9981	0.9975	0.9978	0.9999	0.9924	...	0.9844	0.9968	0.9998	0.9995	0.9996	1.0000	0.9928	0.9933	0.9930	0.9993
3	BiLSTM-char	0.9952	0.9918	0.9935	0.9998	0.9978	0.9960	0.9969	0.9999	0.9945	...	0.9708	0.9964	0.9999	0.9996	0.9997	1.0000	0.9902	0.9933	0.9917	0.9994

4 rows × 57 columns

=== DEV RESULTS ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9774	0.7133	0.8247	0.7125	0.9683	0.9935	0.9807	0.9624	0.9556	...	0.8419	0.7201	0.7957	0.9989	0.8858	0.7950	0.9489	0.9039	0.9211	0.8627
1	MLP (best config)	0.9559	0.7133	0.8170	0.8786	0.9871	0.9978	0.9924	0.9989	0.9566	...	0.9091	0.9643	0.8166	0.9966	0.8977	0.9807	0.9561	0.9174	0.9334	0.9697
2	BiLSTM	0.9119	0.7380	0.8158	0.8981	0.9935	0.9913	0.9924	0.9988	0.9628	...	0.9249	0.9725	0.8433	0.9914	0.9114	0.9843	0.9481	0.9247	0.9347	0.9755
3	BiLSTM-char	0.9416	0.7298	0.8223	0.9065	0.9903	0.9924	0.9913	0.9992	0.9772	...	0.9176	0.9671	0.8534	0.9959	0.9191	0.9932	0.9503	0.9305	0.9382	0.9803

4 rows × 57 columns

=== TEST RESULTS ===

	Model	P(ADJ)	R(ADJ)	F1(ADJ)	PR-AUC(ADJ)	P(ADP)	R(ADP)	F1(ADP)	PR-AUC(ADP)	P(ADV)	...	F1(SCONJ)	PR-AUC(SCONJ)	P(VERB)	R(VERB)	F1(VERB)	PR-AUC(VERB)	Macro-P	Macro-R	Macro-F1	Macro-PR-AUC
0	Baseline	0.9730	0.7660	0.8571	0.7585	0.9723	0.9932	0.9827	0.9662	0.9746	...	0.8904	0.7953	0.7987	0.9976	0.8871	0.7972	0.9523	0.9197	0.9325	0.8811
1	MLP (best config)	0.9551	0.7646	0.8493	0.9037	0.9866	0.9955	0.9910	0.9996	0.9616	...	0.9289	0.9756	0.8223	0.9944	0.9002	0.9762	0.9564	0.9255	0.9386	0.9727
2	BiLSTM	0.9191	0.7859	0.8473	0.9178	0.9977	0.9932	0.9955	0.9997	0.9757	...	0.9565	0.9829	0.8459	0.9880	0.9114	0.9801	0.9511	0.9361	0.9424	0.9767
3	BiLSTM-char	0.9445	0.7926	0.8619	0.9282	0.9899	0.9944	0.9921	0.9996	0.9696	...	0.9135	0.9769	0.8528	0.9944	0.9182	0.9906	0.9523	0.9363	0.9427	0.9816

4 rows × 57 columns